Protocol Audit Report

Prepared by: Jorge Zerpa

Table of Contents

Protocol Summary

A smart contract application for storing a password. Users should be able to store a password and then retrieve it later. Others should not be able to access the password.

Disclaimer

The Jorge Zerpa's team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

We performed a manual research on the code base looking for any possible error on the written code and any logic implementation error by comparing what the code is doing vs what the program is supposed to do (AKA what is on the provided documentation).

Scope

- `src/PasswordStore.sol`

Roles

- `owner`
- `non-owner`

Issues found

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone and no longer private.

Description: All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract.

We show one such method of reading any data off-chain below.

Impact: Anyone can read the private password, severly breaking the functionality of the protocol.

Proof of Concept: The below test case shows how anyone can read the password directly from the blockchain:

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool to get the hex version of the password

```
cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 1 --rpc-url http://127.0.0.1:8545
```

In order, we have: The deployed contract address, the storage slot index of the `s_password' variable (in this case 1) and finally the rpc url of the chain where the contract is deployed.

The third step throws an output like this: 0x6d7950617373776f726400000000000000000000000000000000000000000014 which is the binary/hex representantion of the password value. So now you can just parse from hex to string by using:

```
cast parse-byte32-string 0x6d7950617373776f726400000000000000000000000000000000000000000014
```

And the output will be the "private" password.

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would requiere the uset to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

[H-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password.

Description: The `PasswordStore::setPassword` function is set to be an `external` function, however, the natspec of the function and overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
function setPassword(string memory newPassword) external {
    // HERE there is no access control, anyone can call this and change the password.
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact: Anyone can set/change the password of the contract, severly breaking the contract intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` file:

▶ Details

On the above function we are making a fuzz test to call `setPassword` from different addresses to set a different password each time and then we compare the existing password with the return of `getPassword` to verify that such password was changed for a user different than the owner of the contract.

Recommended Mitigation: Add an access control conditional to the `setPassword` function, for example:

```
if(msg.sender != s_owner) {
    revert PasswordStore__NotOwner();
}
```

Medium

none

Low

none

Informational

none

Gas

none